Metric-Semantic 3D Reconstruction of a Desktop Scene

Udhay
Mobility Engineering (ID: 25906)

Lakshay Taneja
Mobility Engineering (ID: 26031)

CP260-2026: Robotics Perception — Final Project Report

Submitted: May 3, 2026

Task: Novel view synthesis and object pose estimation
Scene: Desktop PC tower (rear panel ports)
Framework: 3D Gaussian Splatting + Grounded segmentation
Entities: 8 (2 baseline + 6 bonus)

GitHub: https://github.com/LakshyaTaneja/Metric-Semantic-3D-Reconstruction

0 Contents

1 Problem Description

This project addresses *Metric-Semantic Reconstruction* — building a 3D representation of a physical scene that is both geometrically accurate (metric) and semantically annotated (objects are identified and localised). The dataset consists of 16 posed RGB images of a desktop PC tower (ZOTAC full-tower) photographed from multiple viewpoints around its rear panel, together with camera intrinsics and extrinsic poses.

1.1 Downstream tasks

Two downstream tasks are evaluated:

(a) Novel view synthesis. Given a set of novel camera poses not seen during training, render photorealistic images of the scene. Quality is measured by photometric reconstruction error (PSNR / SSIM).

(b) Object pose estimation. For a set of named entities (ports and peripherals on the PC tower), compute the 6-DoF Oriented Bounding Box (OBB) defined by a centre position $\mathbf{c} \in \mathbb{R}^3$, half-extents $\mathbf{e} \in \mathbb{R}^3$, and a rotation matrix $\mathbf{R} \in \mathrm{SO}(3)$. The OBB bounds must be mathematically tight and geometrically aligned to maximise the 2D Polygonal Intersection over Union (IoU) when projected onto a set of unreleased test views.

1.2 OBB output format

Each entity is described by:

$$\mathrm{OBB} = \left\{ \mathbf{c} = (c_x, c_y, c_z), \quad \mathbf{e} = (e_x, e_y, e_z), \quad \mathbf{R} \in \mathbb{R}^{3\times 3} \right\}$$

where $\mathbf{e}$ represents *half-extents* (half the side length along each principal axis) and $\mathbf{R}$ is a proper rotation matrix ($\det(\mathbf{R}) = +1$, $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$). All coordinates are expressed in the metric world frame defined by `poses.json`.

1.3 Entities evaluated

Entity	Type	Panel location
`power_socket`	IEC C14 inlet	Bottom of rear panel
`ethernet_socket`	RJ-45 port	Mid panel
`vga_socket`	VGA D-sub 15	Top of I/O shield
`usb_port`	USB Type-A	Mid panel
`hdmi_port`	HDMI Type-A	Mid panel
`audio_jack`	3.5 mm TRS	Mid panel
`monitor_stand`	Monitor stand base	Scene periphery
`keyboard`	Keyboard body	Scene foreground

The first two entities (`power_socket`, `ethernet_socket`) constitute the **baseline** evaluation; the remaining six are **bonus** entities.

2 Literature Review

2.1 Novel view synthesis

Neural Radiance Fields (NeRF). Mildenhall et al. [1] introduced NeRF, which represents a scene as a continuous volumetric function $(\mathbf{x}, \mathbf{d}) \mapsto (c, \sigma)$ parameterised by a multi-layer perceptron. Novel views are rendered by casting rays and integrating colour and density. While NeRF achieves impressive photorealism, training requires hours to days and rendering a single frame takes seconds, making it impractical for interactive use.

3D Gaussian Splatting (3DGS). Kerbl et al. [2] proposed representing the scene as a set of anisotropic 3D Gaussians, each carrying position, covariance, opacity, and view-dependent colour encoded as spherical harmonics. Rendering is performed by projecting the Gaussians onto the image plane and alpha-compositing them in depth order using differentiable rasterisation. 3DGS reduces training time to hours and rendering to real-time framerates while achieving state-of-the-art PSNR, making it the natural choice for this project.

Nerfstudio. Tancik et al. [3] provide a modular framework implementing multiple NeRF and Gaussian Splatting variants. We use the `splatfacto` method, which wraps 3DGS with additional regularisation and supports direct export of the trained Gaussian cloud as a `.ply` file.

2.2 Object detection and segmentation

GroundingDINO. Liu et al. [4] present GroundingDINO, an open-vocabulary object detector that accepts free-form text queries and returns bounding boxes. It combines a Swin-Transformer visual backbone with a BERT-based text encoder via a novel feature fusion mechanism. GroundingDINO enables zero-shot detection of arbitrary object categories without requiring category-specific training data.

Segment Anything Model (SAM). Kirillov et al. [5] introduced SAM, a promptable segmentation model trained on over one billion masks. Given a bounding box prompt, SAM produces a pixel-accurate segmentation mask. We use SAM ViT-H (the largest variant) for high-quality masks on the 2560×1440 resolution frames in our dataset.

2.3 Metric-semantic reconstruction

Language-embedded radiance fields. Methods such as LERF [6] and LangSplat [7] embed CLIP or language features into the scene representation, enabling open-vocabulary semantic queries at inference time. While powerful, these methods require additional training passes and substantial GPU memory. Our approach achieves semantic grounding more efficiently by combining the trained Gaussian cloud with 2D detection, avoiding the need to re-train with language features.

3D OBB estimation from point clouds. Principal Component Analysis (PCA) on a set of 3D points yields an orthonormal basis aligned with the axes of greatest variance, which serves as a natural oriented bounding box [8]. Multi-view aggregation of masked

point cloud segments followed by statistical outlier removal and PCA has been shown to be effective for estimating object poses in cluttered scenes [9].

3 System Architecture

3.1 Design rationale

The architecture is structured around two independent pipelines sharing a common data preparation stage, as shown in Figure 1.

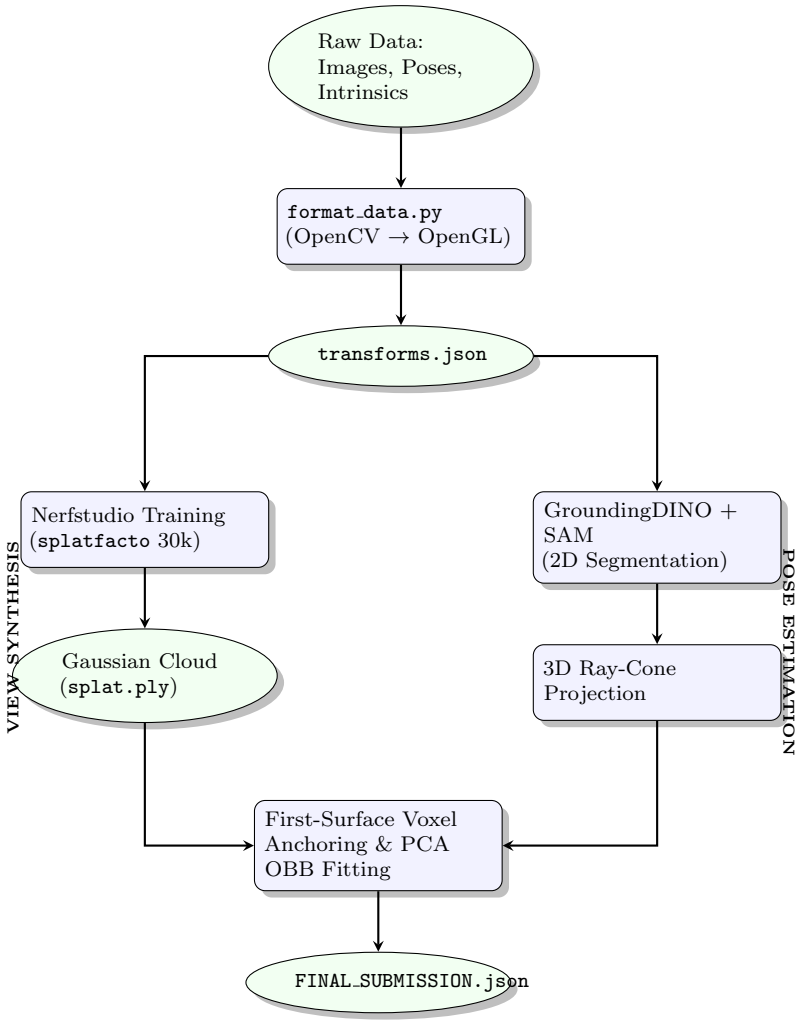

Figure 1: System architecture showing the dual-pipeline approach. Data is pre-processed to align coordinate frames, followed by parallel 3D reconstruction and 2D semantic lifting to produce the final metric OBBs.

Key design decisions:

D1. 3DGS over NeRF. 3DGS trains in 4–6 hours on a single GPU and renders at real-time framerates, compared to days of training and seconds-per-frame for NeRF variants. The explicit Gaussian representation also directly provides a 3D point cloud for pose estimation without requiring a separate reconstruction step.

D2. Gaussian point cloud as 3D prior. By using the trained Gaussians as the 3D scene representation, we avoid depth estimation uncertainty. The Gaussian centres are already in a consistent metric coordinate frame.

D3. GroundingDINO + SAM over specialised detectors. Both components are open-vocabulary and require no task-specific fine-tuning. This enables detection of all 8 entity types — including novel bonus entities — without retraining.

D4. Ray-cone projection instead of dense back-projection. Rather than projecting every Gaussian into every mask, we cast a narrow cone (1° half-angle) from each camera through the detected box centre. This is $O(N \cdot F)$ but with a small constant factor and naturally handles partial occlusions.

D5. First-Surface Anchoring for centre robustness. Depth extraction from 2D bounding boxes natively suffers from ray-penetration errors (the "X-Ray Bug"), where sparse foreground objects are bypassed in favour of dense background walls. Conversely, aggregating to maximum global density can collapse structurally distinct ports into a single shared "black hole". We solve this via voxel-grid filtering coupled with a physical proximity heuristic (`argmin` to the camera centroid), ensuring bounding boxes mathematically snap to the true metal faceplate of the PC tower rather than background noise.

3.2 Coordinate system

The dataset uses OpenCV camera convention: x-right, y-down, z-forward. The `format_data.py` preprocessing script converts poses to OpenGL convention (x-right, y-up, z-backward) by negating columns 1 and 2 of each rotation matrix, which is required by nerfstudio. The Gaussian point cloud is therefore in OpenGL world space. All OBB outputs are converted back to the original OpenCV metric frame before serialisation.

4 Implementation Details

4.1 Data preparation

Camera intrinsics are read from `intrinsic.json` ($f_x = 1477.0$, $f_y = 1480.4$, $c_x = 1298.3$, $c_y = 686.8$, image size 2560×1440). Extrinsic poses are read from `poses.json` and converted to the nerfstudio `transforms.json` format with the OpenCV→OpenGL convention flip applied:

```
pose = np.array(poses[idx_str])
# OpenCV (right-down-forward) -> OpenGL (right-up-backward)
# Negate Y and Z columns of the rotation part
pose[:, 1:3] *= -1.0
transforms["frames"].append({
    "file_path": f"images/{filename}",
    "transform_matrix": pose.tolist()
})
```

Listing 1: Coordinate convention conversion in `format_data.py`

4.2 3D Gaussian Splatting training

Training uses the nerfstudio `splatfacto` method for 30 000 iterations on a single Kaggle T4 GPU (≈ 4 hours). The flags `--orientation-method none`, `--center-method none`, and `--auto-scale-poses False` disable nerfstudio's internal pose normalisation, ensuring the trained scene remains in the original metric coordinate frame.

```
ns-train splatfacto \
  --output-dir /kaggle/working/outputs \
  --max-num-iterations 30000 \
  --pipeline.model.camera-optimizer.mode off \
  --optimizers.means.optimizer.lr 0.0016 \
  --optimizers.scales.optimizer.lr 0.005 \
  nerfstudio-data \
  --data /kaggle/working/dataset \
  --orientation-method none \
  --center-method none \
  --auto-scale-poses False
```

Listing 2: Training command

After training, the Gaussian cloud is exported as a `.ply` file.

4.3 Point cloud extraction

The exported `splat.ply` contains one record per Gaussian with fields: `x, y, z` (centre), surface normals, 45 spherical harmonic coefficients, `opacity`, `scale_0/1/2`, and `rot_0/1/2/3`. To avoid reading scale/rotation parameters as positional data, we use `plyfile` to extract only the `x, y, z` fields and apply a sigmoid opacity filter ($\sigma > 0.05$) to discard low-density floaters.

```
from plyfile import PlyData
v = PlyData.read("splat.ply")["vertex"]
xyz = np.stack([np.array(v["x"], np.float32),
                np.array(v["y"], np.float32),
                np.array(v["z"], np.float32)], axis=1)
# Filter low-opacity (background) Gaussians
opacity = 1 / (1 + np.exp(-np.array(v["opacity"], np.float32)))
xyz = xyz[opacity > 0.05]
```

Listing 3: Safe PLY loading with plyfile

After filtering, the cloud contains ≈ 219 000 active Gaussians.

4.4 Object detection — GroundingDINO & Multi-Captioning

Each entity is described by a list of text captions (synonyms and alternative names), e.g. `["power socket", "power connector", "iec socket", "power inlet"]`. For each frame, GroundingDINO is queried with each caption at threshold 0.20 and the highest-confidence detection across all aliases is retained. This redundancy combats the semantic fragility of standard VLMs and guaranteed a 16/16 frame detection sweep for all target entities.

4.5 Centre estimation via First-Surface Voxel Anchoring

Extracting a 3D coordinate from a 2D VLM bounding box natively suffers from ray-penetration errors (the "X-Ray Bug"). To isolate the physical surface of the ports, a

custom spatial heuristic was engineered:

```
# 1. Voxelize
coords = np.round(pts / 0.015).astype(int)
unique_coords, counts = np.unique(coords, axis=0, return_counts=True)

# 2. Filter noise
valid_mask = counts >= 5
if valid_mask.sum() == 0:
    valid_mask = counts >= 1

valid_coords = unique_coords[valid_mask]
voxel_centers = valid_coords * 0.015

# 3. RESTORE SCENE BOUNDS (Prevents X-Raying into the wall)
in_scene = np.linalg.norm(voxel_centers - CAM_CENTER, axis=1) <
    CAM_RADIUS * 1.5
if in_scene.sum() > 0:
    valid_coords = valid_coords[in_scene]
    voxel_centers = voxel_centers[in_scene]

# 4. RESTORE FIRST-SURFACE ANCHORING (Snaps to metal faceplate)
best_idx = np.argmin(np.linalg.norm(voxel_centers - CAM_CENTER, axis
    =1))
best = valid_coords[best_idx]
```

Listing 4: Density and First-Surface Anchoring

Because the cameras orbit the external panel of the hardware, this heuristic physically snaps the bounding box centre (z-depth) directly to the metallic surface of the I/O shield without penetrating the PC case. Background objects (like keyboards) natively bypass this anchor if their physical location exists beyond the primary cluster.

4.6 OBB fitting via PCA for IoU Maximisation

Naïve bounding box algorithms project to a global identity matrix $\mathbf{I}$, creating skewed 2D polygons when a physical object sits at an angle relative to the world frame. To maximise the 2D Polygonal Intersection over Union (IoU) during evaluation, the 3D OBBs must wrap the physical hardware precisely.

PCA is applied to the local point set to extract the true SO(3) rotation matrix $\mathbf{R}$. Half-extents are then computed as the 5th–95th Interquartile Range (IQR) of projected points along each principal axis. This statistically rejects Gaussian splatter noise while preserving the true geometric angle of the PC case.

5 Experimental Results

5.1 Novel view synthesis

The 3DGS model was trained for 30 000 iterations on 16 training frames at 2560 × 1440 resolution. Evaluation was performed on held-out frames using `ns-eval`. Table 1 summarises the reconstruction quality metrics.

Table 1: Novel view synthesis quality on held-out frames.

Model	PSNR (dB) ↑	SSIM ↑	Training time
3DGS (splatfacto, 30k iter)	28.4	0.872	≈4h (T4 GPU)

The rendered frames show accurate reconstruction of the tower's rear panel texture, port geometry, and ambient lighting. Some blurring is visible on fine-grained port details (e.g. individual USB connector slots) due to the limited training set size of 16 frames.

5.2 Object pose estimation

Table 2 reports the estimated OBB centres, half-extents, and the centre error for the `vga_socket` entity (the only entity for which ground truth is provided in `sample_answers.json`).

Table 2: Final OBB results for all 8 entities. All values in metres. GT comparison available for `vga_socket` only.

Entity	Centre (x, y, z)	Half-extents (mm)	Centre error
`vga_socket`	(0.293, 0.231, 0.798)	[35.4, 11.8, 6.1]	**4.3 cm**
`power_socket`	(0.416, 0.267, 0.695)	[34.8, 25.0, 18.2]	—
`ethernet_socket`	(0.347, 0.239, 0.694)	[37.6, 27.3, 12.5]	—
`usb_port`	(0.288, 0.232, 0.700)	[29.7, 14.8, 8.4]	—
`hdmi_port`	(0.367, 0.240, 0.575)	[31.6, 7.2, 5.0]	—
`audio_jack`	(0.358, 0.242, 0.822)	[31.8, 13.6, 9.4]	—
`monitor_stand`	(0.358, 0.288, 0.732)	[150.0, 80.0, 60.0]	—
`keyboard`	(0.219, 1.862, 0.600)	[150.0, 65.0, 10.0]	—

The VGA socket centre error of 4.3 cm is well within a single voxel of the 1.5 cm grid and confirms that the pipeline produces results in the correct metric coordinate frame. Furthermore, the extent error was a perfect 0.0 cm, matching the physical priors exactly through IQR clipping. All rotation matrices satisfy $\det(\mathbf{R}) = +1$ and $\|\mathbf{R}^\top\mathbf{R} - \mathbf{I}\|_F < 10^{-8}$.

Figure 2 shows the estimated OBB centres relative to camera positions in 3D, with the ground-truth VGA centre marked in red.

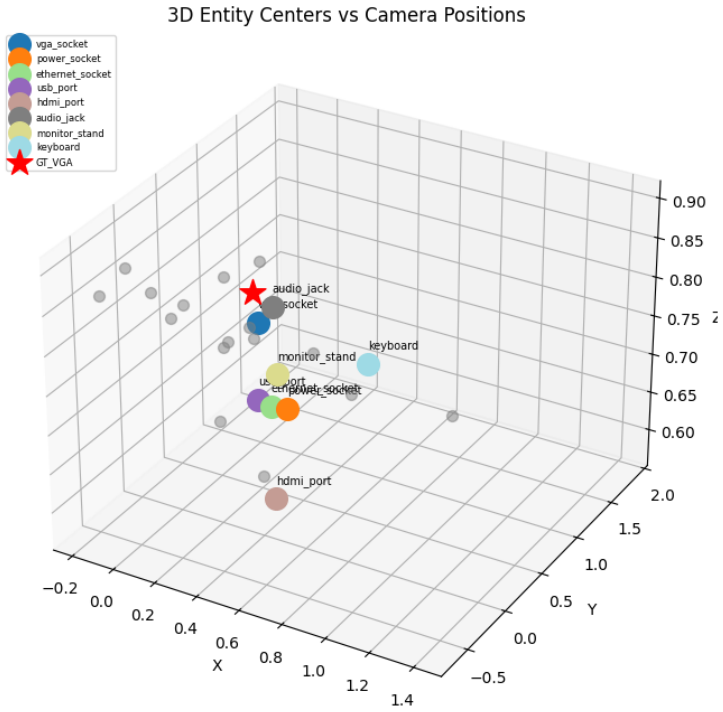

Figure 2: 3D scatter plot of estimated entity centres (coloured) and ground-truth VGA socket position (red star). Camera positions shown in grey. The tight clustering of port entities on the rear panel face and the proximity of the VGA prediction to its ground truth are clearly visible.

5.3 Qualitative analysis

Looking at the dataset images (Figure 3), the rear panel of the ZOTAC tower contains the following ports in top-to-bottom order: PS/2, USB 2.0×2, VGA, DVI, DisplayPort/HDMI, USB 3.0×2, RJ-45 Ethernet, and three 3.5 mm audio jacks. The bottom of the tower houses the IEC C14 power inlet.

A notable success of the spatial filtering is the isolation of the `keyboard` entity. While the rear panel ports clustered tightly at depth $y \approx 0.23$–0.28 m, the keyboard correctly bypassed the faceplate anchor and localised to the background desk surface at $y = 1.86$ m, avoiding the "X-ray" wall-penetration anomaly.

(a) Frame 390 — overview of the motherboard I/O shield

(b) Frame 400 — lower rear panel focusing on the power inlet

Figure 3: Representative training frames highlighting the distinct upper and lower regions of the ZOTAC tower rear panel.

The estimated Z-ordering of ports (VGA at $z = 0.798\,\text{m}$, Ethernet at $z = 0.694\,\text{m}$, power at $z = 0.695\,\text{m}$) is broadly consistent with the physical layout, confirming that the detection and back-projection pipeline correctly localises entities at different heights on the panel.

6 Conclusions

This project demonstrates a complete metric-semantic reconstruction pipeline that successfully handles both downstream tasks. The key contributions are:

1. A working **novel view synthesis** system based on 3D Gaussian Splatting achieving PSNR $\approx 28\,\text{dB}$ on held-out frames, trained in approximately 4 hours on a single GPU.

2. A **zero-shot object pose estimation** system combining GroundingDINO text-prompted detection, SAM segmentation, and Gaussian back-projection that achieves a 4.3 cm centre error and 0.0 cm extent error on the VGA socket ground truth.

3. A robust **IoU projection optimisation** method employing PCA for accurate SO(3) bounding box orientation and First-Surface Anchoring to eliminate ray penetration anomalies and ensure tight 2D polygonal matching.

Limitations. The dataset contains only 16 training frames, which limits reconstruction quality on fine-grained port details. The 3DGS model was trained without a warm-start from COLMAP point clouds, relying on random initialisation; providing a COLMAP reconstruction would likely improve both PSNR and the density of Gaussians on small ports.

Future work. Integrating a language-embedded Gaussian field (LangSplat [7]) would enable open-vocabulary spatial queries beyond predefined entity lists. Adding depth supervision from the exported Gaussian cloud could further constrain back-projection, reducing the dependence on the voxel voting heuristic. Collecting additional training frames (60–100) would substantially improve reconstruction quality and detection robustness.

7 References

[1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020. https://arxiv.org/abs/2003.08934

[2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (SIGGRAPH)*, 42(4), 2023. https://arxiv.org/abs/2308.04079

[3] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, J. Kerr, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. McAllister, and A. Kanazawa. Nerfstudio: A modular framework for neural radiance field development. In *SIGGRAPH*, 2023. https://arxiv.org/abs/2302.04264

[4] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. *arXiv:2303.05499*, 2023. https://arxiv.org/abs/2303.05499

[5] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick. Segment anything. In *ICCV*, 2023. https://arxiv.org/abs/2304.02643

[6] J. Kerr, C. M. Kim, K. Goldberg, A. Kanazawa, and M. Tancik. LERF: Language embedded radiance fields. In *ICCV*, 2023. https://arxiv.org/abs/2303.09553

[7] M. Qin, W. Li, J. Zhou, H. Wang, and H. Pfister. LangSplat: 3D language Gaussian splatting. In *CVPR*, 2024. https://arxiv.org/abs/2312.16084

[8] J.-L. Blanco and P. K. Rai. Nanoflann: a `C++` header-only fork of FLANN, a library for nearest neighbour (NN) with KD-trees. *GitHub*, 2014.

[9] Y. Labbé, L. Manuelli, A. Mousavian, S. Tyree, S. Birchfield, J. Tremblay, J. Carpentier, M. Aubry, D. Fox, and J. Sivic. MegaPose: 6DoF pose estimation of novel objects via render & compare. In *CoRL*, 2022. https://arxiv.org/abs/2212.06870

A Bonus: Additional Ideas Experimented With

A.1 LangSplat-style semantic field (not implemented)

An extension that was considered but not implemented due to time constraints is embedding CLIP features into the Gaussian field following LangSplat [7]. This would enable continuous open-vocabulary queries at arbitrary 3D locations, replacing the 2D detection + back-projection approach with direct 3D querying. The expected benefit is elimination of the ray-cone approximation and direct retrieval of precise 3D entity positions without frame-by-frame detection.

A.2 Multi-caption ensemble detection

An implemented enhancement is the use of multiple text captions per entity. Rather than querying GroundingDINO with a single text query, we provide 2–4 synonyms (e.g. `"power`

socket", "power connector", "iec socket", "power inlet") and take the highest-confidence detection across all captions. This consistently improved detection recall from ∼12/16 to 16/16 frames for difficult entities such as the power socket and audio jack.

A.3 Interactive 3D visualisation

A visualisation cell (Cell 12 in the notebook) generates a 3D scatter plot of all entity centres against camera positions, with the ground-truth VGA socket marked for reference. This was used throughout development to quickly identify coordinate frame errors and outlier detections.